

Two Runtimes, One Verdict: Boundary-Verified ONNX/NumPy Serving for Streaming Intrusion Detection at the Edge

Thai Nguyen Vu¹, Dung Van Bui^{2,3}, Nhut Tri Do^{2,3}, and Van Du Nguyen⁴

¹ Campus in Ho Chi Minh City, University of Transport and Communications, Ho Chi Minh City, Vietnam

thaivn_ph@utc.edu.vn

² Vietnam National University Ho Chi Minh City, Ho Chi Minh City, Vietnam

³ University of Information Technology, Ho Chi Minh City, Vietnam

dungbv.17@grad.uit.edu.vn, trinhutdo@uit.edu.vn

⁴ Faculty of Information Technology, Nong Lam University, Ho Chi Minh City, Vietnam

nvdu@hcmuaf.edu.vn

Abstract An edge intrusion detector that ships a primary and a fallback runtime promises the same verdict from both. We study that promise on a two-stage Kafka pipeline on Jetson Orin Nano Super boards whose 200-tree classifier is served by ONNX Runtime or by a NumPy engine exported from the same ONNX graph. Replaying recorded flows through both runtimes is a weak test of it, because recorded flows almost never sit on a split threshold, the only place tree runtimes can disagree. We instead build one input per split node that lands on its threshold, plus its two float32 neighbours: 287,223 probes covering all 95,741 nodes. A float64 NumPy traversal flips 328 labels against ONNX Runtime on the probes but only 15 on the 447,275-row CICIDS2017 test split, so a 2,000-row replay gate misses it 93.5% of the time; the deployed float32 traversal flips none. Flips reach the label because the forest’s splits share only 815 distinct thresholds, so one boundary value moves many trees at once. We also make the fallback usable at streaming batch sizes with a block-lockstep traversal ($71\times$ at batch 1, $11\times$ at 20, $0.9\times$ at 2,000), and add an early-exit rule that provably keeps the full forest’s label: gate-forwarded traffic needs 104 of 200 trees, a $2.0\times$ saving at the rule’s ceiling. Frozen and run on CSE-CIC-IDS2018 traffic, the detector collapses (F1 0.98 to 0.24) while both serving guarantees hold on all 477,771 shifted flows, and the share of flows early exit cannot settle rises from 4.8% to 28.2%, a label-free drift signal the serving path computes anyway. [*Jetson re-measurement of the traversal tables and wired-link latency are pending.*]

Keywords: Intrusion detection · Edge computing · ONNX Runtime · Differential testing · Random forest · Apache Kafka.

1 Introduction

Streaming ML-based intrusion detection increasingly runs at the network edge to cut bandwidth and response time [20, 8]. A recurring failure mode in such deployments is not the model but the serving stack around it: a framework convenient for offline training

is reused for online serving because it is operationally simpler to run one stack than two, and that convenience carries a memory and latency cost that a constrained ARM board cannot always absorb. Earlier deployment work on this project’s two-stage anomaly-gate-then-classifier pipeline exposed exactly this limitation: the classifier was accurate, but its serving path was heavier than the 8 GB Jetson Orin Nano Super boards it ran on could comfortably sustain.

The response to that limitation was ONNX-EDGEIDS⁵, a serving toolkit organized around one portable artifact, an ONNX `TreeEnsembleClassifier` graph, with two interchangeable runtimes behind it. The primary runtime is ONNX Runtime; a second, dependency-minimal runtime re-implements the same forest in pure NumPy for boards where no matching `onnxruntime` wheel exists. Critically, the NumPy exporter does not depend on the framework that produced the ONNX graph: it parses the graph’s standard tree-ensemble operator attributes (split features, thresholds, per-leaf class weights) rather than a framework-specific internal representation, so the same exporter works on an ONNX tree ensemble regardless of its origin. A companion validation script treats agreement between the two runtimes as a release gate rather than a one-off sanity check, failing the build if label agreement or confidence drift falls outside tolerance.

We integrate both runtimes into a two-stage Kafka pipeline – an anomaly gate on one Jetson board forwarding suspicious flows to a classifier on a second – and then ask three questions that the integration itself raises. *Do the two runtimes really agree?* Replaying recorded flows says yes, but recorded flows almost never sit on a split threshold, which is the only place two tree runtimes can disagree; we probe every threshold directly instead (Sect. 4.3). *Why is the NumPy fallback slow at the small batches a streaming node serves?* Because it pays Python dispatch once per tree per level; advancing blocks of trees in lockstep removes that (Sect. 6.4). *How much of the forest does forwarded traffic need?* Barely more than half, under a rule that cannot change a label (Sect. 6.5).

Contributions. (1) *Boundary-directed parity testing* for tree-ensemble serving artifacts: one probe per split node, on the threshold and one float32 step either side, with a path-satisfying construction that reaches 99.9% of nodes. On the deployed forest it exposes a float64/float32 traversal divergence $34\times$ more densely than replaying the whole labelled test split, and shows that a 2,000-row replay gate would miss it 93.5% of the time; (2) The observation that explains why such divergences flip *labels*, not just one tree’s vote: histogram-trained forests reuse thresholds (815 distinct pairs for 95,741 splits here), so arithmetic disagreement at one value is correlated across the ensemble rather than averaged out by it – and the same fact makes an exhaustive boundary gate cheap; (3) A block-lockstep NumPy traversal, measured on real flows against the per-tree loop it replaces, including the batch size at which it stops winning; (4) An exact early-exit rule for the binary forest and its measured cost on gate-forwarded traffic; (5) A shift stress test that separates what drift breaks (the detector and the gate) from what it cannot (runtime parity and early-exit exactness), and shows the early-exit undecided rate acting as a label-free drift signal; (6) A framework-agnostic ONNX-to-NumPy exporter,

⁵ <https://github.com/vuthainguyen1602/onnx-edge-ids>

on-board measurements of both runtimes, and the experiment code, artifacts and result files behind every number in this paper⁶.

2 Related Work

Edge intrusion detection. Single-board deployments of ML-based IDS are now common, from early Raspberry Pi systems [18] to recent ensemble and metaheuristic-optimized detectors on IoT-edge and industrial-control hardware [2, 16, 22] and autoencoder- or generative-model-based detection on Jetson boards [9, 3]. These studies optimize the detector; the present paper instead treats the *serving stack* as the object of study, on the observation (from earlier deployment work on this system) that model accuracy and serving cost are separate axes that a single evaluation can conflate.

Serving classical ML on constrained hardware. ONNX⁷ standardizes a small operator set, including `TreeEnsembleClassifier` for decision-tree ensembles, so a model trained in one framework can be served by any ONNX-compatible runtime; ONNX Runtime⁸ is the reference CPU/GPU implementation and is what we target as the primary backend. Nakandala et al. [15] make a related argument for tree ensembles specifically: compiling scikit-learn-style pipelines into tensor operations lets a GPU tensor runtime, rather than a tree-optimized one, serve them, with large reported speedups over native execution on batched workloads. Our NumPy fallback takes the opposite trade-off deliberately: no compilation step and no GPU dependency, at the cost of the batch-size sensitivity this trades in. Koschel et al. [11] show that this cost is not fundamental to ARM hardware: porting the QuickScorer tree-traversal algorithm from Intel AVX to ARM NEON yields up to 9.4× speedups for ensemble inference on IoT-class ARM devices, with fixed-point quantization improving throughput further at a small accuracy cost. We do not port QuickScorer’s bitvector algorithm or write NEON intrinsics; we do take its underlying lesson – evaluate many trees in lockstep rather than one at a time – and apply it at the NumPy level to `NumpyInferenceEngine`’s traversal loop, with a measured result rather than a citation (Sect. 6.4).

ONNX and quantization in network intrusion detection. Model compilation and quantization for ONNX-served detectors is an active line of work in this application domain specifically: Eren et al. [5] report a systematic comparison of ONNX compilation and quantization configurations for DrDoS/amplification-attack detectors (MLP, CNN, GRU architectures), showing that quantized ONNX export can materially change throughput and latency for network-security classifiers without simply assuming DNN-quantization results transfer unchanged. Their target model family is different from the tree ensemble served here, but the result establishes that ONNX-level quantization is a validated, actively studied lever in exactly this application space, and motivates evaluating it against our tree-ensemble graph rather than the deep architectures it was demonstrated on.

⁶ <https://github.com/vuthainguyen1602/onnx-edge-ids-paper> (directory code/); software archive: <https://doi.org/10.5281/zenodo.22731927>.

⁷ <https://onnx.ai/>

⁸ <https://onnxruntime.ai/>

Two-stage detection, ensemble early exit, and runtime equivalence. Cheap-filter-then-forest cascades are now routine in intrusion detection, including on embedded targets [6]; our pipeline has that shape and we claim nothing new for it. Stopping an ensemble before every member has voted is also old: Hernández-Lobato et al. [10] halt once the remaining votes are statistically unlikely to change the outcome, and Wang et al. [21] learn an evaluation order and per-step thresholds jointly. Both trade a small, tunable disagreement for speed. An intrusion detector with two runtimes cannot take that trade without making the runtimes disagree with each other, so Sect. 6.5 uses the exact counterpart – stop only when the unevaluated trees cannot flip the label – and measures what the exactness costs. On equivalence, differential testing of numerical libraries is well established for deep-learning APIs, where boundary and non-finite inputs are known to expose inconsistent comparison semantics [4], and Naik et al. [14] analyze symbolically how far a tree ensemble’s output can move under small input perturbations. The precision sensitivity of tree thresholds under float32 export is documented by the converter projects themselves. What we have not found is this applied where it bites operationally: as a release gate for two *serving* runtimes of one deployed detector, with probes derived from the model’s own split table rather than from recorded or random inputs.

Evaluation protocol. Arp et al. [1] and Lanvin et al. [12] document how easily an offline security-ML evaluation is inflated by leakage or dataset artifacts; the classifier served by this toolkit was selected under a leakage-aware, deduplicated CICIDS2017 [19] protocol with SHAP-based [13] feature selection in earlier work on this system, out of scope for the present paper. What is in scope here is a different correctness question: whether the *exported serving artifacts* reproduce the classifier they were exported from. Sect. 4 treats that as a release-gated, measured property rather than an assumption.

3 System Architecture

3.1 Two-Stage Kafka Pipeline

Fig. 1 shows the deployment topology. A coordinator host runs Kafka and replays CICIDS2017-derived flow records onto an input topic at a configurable rate. Jetson #1 runs the anomaly gate: a scikit-learn autoencoder and scaler (loaded via `joblib`, trained offline and out of scope here) score each batch, and flows whose reconstruction error clears a calibrated threshold are forwarded to a suspicious-flow topic with their anomaly score attached. Jetson #2 consumes that topic and classifies each flow with the swappable `InferenceEngine` described below. Both gate and classifier batch their Kafka consumption with the same policy: a batch is served either once it reaches a target size or once a linger timeout elapses since its first record, whichever comes first, so a thin stream still gets bounded latency instead of waiting indefinitely to fill a batch.

3.2 Swappable Classifier Tier

- `FeatureMatrixBuilder` turns a batch of Kafka flow dicts into a dense float32 matrix in a fixed column order, mapping missing/NaN/Inf values to zero.

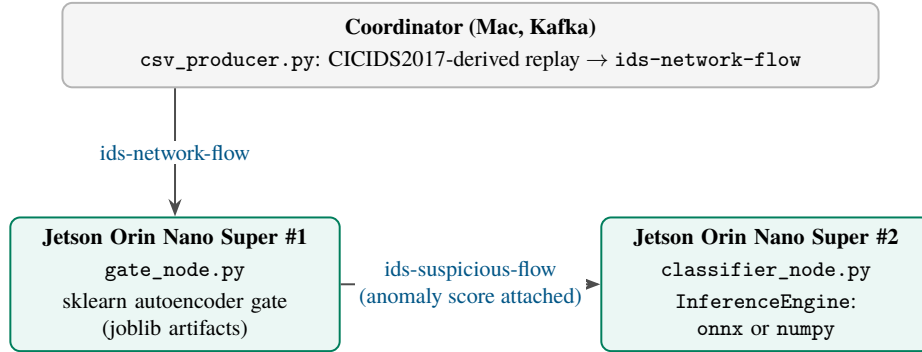


Fig. 1. Two-stage Kafka pipeline: gate on Jetson #1, swappable ONNX/NumPy classifier on Jetson #2. This minimal toolkit prints per-batch metrics and an optional CSV; the PostgreSQL/alerting/Grafana stack of the earlier deployment work is out of scope here.

- **InferenceEngine** is the shared contract: `create_inference_engine(engine, model_path, providers)` returns either backend, and both return a common `BatchPrediction` (predictions, per-row confidences, batch timing/attack-count stats), so `classifier_node.py` never branches on which backend is active.
- **OnnxInferenceEngine** loads the `.onnx` artifact with ONNX Runtime’s CPU execution provider (GPU/TensorRT providers are selectable but not evaluated in this paper).
- **NumpyInferenceEngine** loads the flattened forest described in Sect. 4 and walks every tree with a vectorized loop over an active-row boolean mask: at each step, active rows look up their current node’s split feature and threshold, move left or right, and rows that land on a leaf accumulate that leaf’s class scores and drop out of the mask.

Fig. 2 shows the contract. Selecting a backend is a single `--engine onnx|numpy` flag to `classifier_node.py`; nothing else in the pipeline changes.

4 Model Export and Conversion Correctness

4.1 Framework-Agnostic NumPy Export

The NumPy exporter (`scripts/export_numpy.py`) treats the ONNX file as the single source of truth: it loads the graph with the `onnx` package and reads the `ai.onnx.ml.Scaler` node’s `offset/scale` attributes and the `ai.onnx.ml.TreeEnsembleClassifier` node’s `nodes_treeids`, `nodes_nodeids`, `nodes_featureids`, `nodes_values`, `nodes_modes`, `nodes_truenodeids`, `nodes_falsenodeids`, `classlabels_int64s`, `class_treeids`, `class_nodeids`, `class_ids`, and `class_weights` attributes – all part of the ONNX operator specification, not of any particular training framework’s internal format. It currently supports the `BRANCH_LEQ` split mode only and raises rather than silently mis-converting on any other mode, so a future export from a framework using a different split convention fails loudly instead of producing a plausible-looking

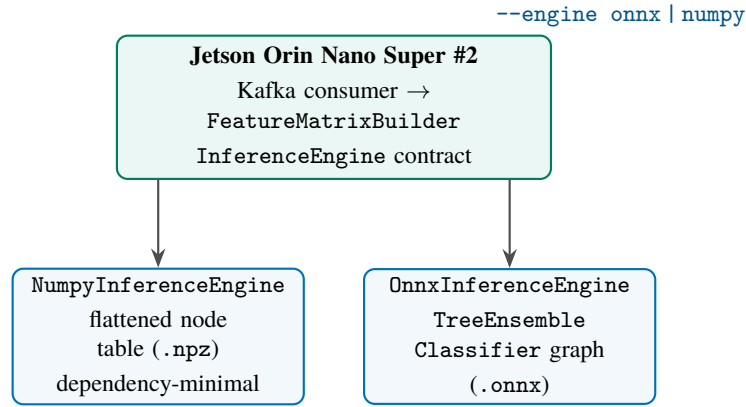


Fig. 2. Swappable classifier tier on Jetson #2: one feature-matrix builder, one `InferenceEngine` contract, two backends selected by a single flag.

but wrong artifact. Because the exporter never touches how the ONNX graph was produced, the same code exports a NumPy fallback for any ONNX tree ensemble that uses this operator, independent of its training framework.

A second correctness check is feature-order agreement: if the source ONNX model’s `doc_string` carries a JSON `feature_columns` list (as this project’s exporter embeds at ONNX-export time), `export_numpy.py` verifies it matches the externally supplied `feature_columns.json` and refuses to export on a mismatch, since a silently permuted feature order would produce confident, wrong predictions rather than an obvious failure.

4.2 Parity as a Release Gate

`scripts/validate_parity.py` runs both backends over the same batch-built feature matrix and reports label agreement and the maximum confidence deviation between them; it exits non-zero if agreement falls below a configurable threshold (default: exact, 1.0) or the deviation exceeds a tolerance (default 10^{-5}), so it is usable as a CI-style gate rather than only a manual check, and it optionally writes a machine-readable JSON report. On a validation run of 2,000 replayed CICIDS2017-derived flows, the two backends agreed on every label (2,000/2,000) with a maximum confidence deviation of 0.0, i.e. bit-identical decisions on the sampled flows; Table 1 reports this alongside artifact size.

4.3 Boundary-Directed Parity Testing

Two tree runtimes fed the same row can only disagree if, at some split, they place the row on different sides of the threshold. That can only happen when the scaled feature value is within rounding distance of the threshold, and recorded traffic is almost never there. Replay therefore tests the runtimes where they cannot differ. We test them where they can.

Table 1. Export validation: NumPy-vs-ONNX prediction parity on a replay sample.

Metric	Value
Replay rows validated	2,000
Label agreement	2,000 / 2,000 (100.0000%)
Max confidence deviation	0.0
ONNX artifact size	7.38 MB
NumPy artifact size	1.23 MB (6.0× smaller)

Source: `scripts/validate_parity.py` on a single replay run (this software’s own submission-checklist validation run); re-run on a held-out sample as a submission gate rather than treated as a one-time result.

For each of the forest’s 95,741 internal nodes, with split feature f and threshold t in scaled space, we (i) walk from the node to its root, intersecting the constraints each ancestor split places on each feature, and write a value from inside every resulting interval into a real test-split row, so the row is routed to the node; (ii) overwrite feature f with the raw float32 value that the scaler maps onto t , and with its float32 predecessor and successor. This yields $3 \times 95,741 = 287,223$ probes. Re-tracing each probe through the forest confirms that 95,664 of the 95,741 (99.9%) arrive at their target node; the remainder have path intervals narrower than float32 can represent after inverse scaling.

Table 2. Label disagreements with ONNX Runtime: boundary-directed probes vs. replaying the whole labelled test split.

Input set	Inputs	NumPy float32	NumPy float64
Probe on threshold	95,741	0	143
Probe one float32 step below	95,741	0	7
Probe one float32 step above	95,741	0	178
All boundary probes	287,223	0	328
Replay, full test split	447,275	0	15

Counts are label flips, not probability differences. Source: `code/results/exp2_boundary_parity.json`; counts do not depend on the machine.

Table 2 compares two NumPy traversals against ONNX Runtime. They differ in one line: the dtype in which the scaler and the threshold comparison are carried out. ONNX’s tree-ensemble operator holds float32 thresholds and ONNX Runtime scales and compares in float32; a traversal that does the same arithmetic in float64 computes a slightly different scaled value and can land on the other side of a threshold it was nominally equal to. The float64 variant is not a strawman: it is the natural default in NumPy, it is what an earlier version of our exporter wrote, and it passes a 2,000-row replay gate almost every time. At the measured replay disagreement rate of $15/447,275$, the chance that a 2,000-row replay contains no disagreeing row is $(1 - 3.35 \times 10^{-5})^{2000} = 93.5\%$. The boundary probes surface the same defect at a rate of $328/287,223$, $34\times$ denser, and deterministically: the probe set is a function of the model, not of what traffic happened

to be recorded. The float32 traversal we deploy produces zero flips on all 287,223 probes and on all 447,275 replayed rows.

Why labels flip, not just votes. A single misrouted tree moves one vote in 200, which should almost never change a label. It does here because the trees are not independent in where they split. The forest’s 95,741 splits use only 815 distinct (feature, threshold) pairs – about 117 nodes per pair – as any trainer that bins each feature into a fixed set of candidate thresholds will produce. A value sitting on one such threshold sits on it in every tree that uses it, so an arithmetic disagreement there is correlated across the ensemble instead of being averaged out by it. For a detector this is more than a numerical curiosity: in a fleet where some boards run the primary runtime and others the fallback, a flow whose features sit on shared thresholds receives different verdicts depending on which board sees it. The same redundancy makes the gate cheap: 815 distinct thresholds, not 95,741 nodes, bound the number of boundary values that need probing.

5 Experimental Setup

The testbed is two NVIDIA Jetson Orin Nano Super Developer Kits (6-core Arm Cortex-A78AE CPU, Ampere GPU, 8 GB LPDDR5) and a coordinator host running Kafka, over a LAN; at the time of writing, Jetson #2’s inter-board link is Wi-Fi rather than wired, which affects only the end-to-end transport figures still marked pending below, not the in-process measurements in Sect. 6. `scripts/csv_producer.py` replays CICIDS2017-derived [19] flow rows onto the input topic at a configurable rate; `gate_node.py` runs on Jetson #1 and `classifier_node.py` on Jetson #2, the latter re-run once per backend (`--engine onnx` and `--engine numpy`) with the gate held fixed, and once more across a batch-size sweep (1, 20, 500) to characterize batch sensitivity. Reported metrics: throughput (rows/s), p95 per-row latency (ms), resident memory (RSS), board power (W, `tegrastats`, not idle-subtracted, so figures are total-board rather than active-only), cold-start time to first verdict, and the parity metrics of Sect. 4.2.

Off-board experiments. The boundary-parity, traversal, early-exit and shift experiments (Tables 2, 5, 6, 7) run the same served artifacts from `code/` on an ARM64 development machine (Apple M3, NumPy 2.5, ONNX Runtime 1.30, CPU provider), not on the boards. They use the labelled CICIDS2017 test split (447,275 flows, 15.0% attacks) and, for the shift experiment, the CSE-CIC-IDS2018 test split prepared identically (477,771 flows, 12.7% attacks). Disagreement and label-identity counts do not depend on the machine; throughput does, and is labelled as such wherever it appears.

6 Results

6.1 Steady-State Cost per Backend

Table 3 reports classifier-node cost at batch size 20 for each backend, with the gate’s own cost reported separately since it runs a different model (an autoencoder, not the forest). ONNX Runtime sustains roughly $133\times$ the NumPy backend’s throughput at

this batch size and a fraction of its p95 latency, at the cost of higher resident memory and marginally lower measured board power (4.64 vs. 4.93 W; since power is not idle-subtracted, this difference should be read cautiously rather than as a per-inference energy claim). The NumPy row here is the original per-tree traversal, measured on the Jetson boards; Sect. 6.4 replaces that traversal and measures the replacement on different (ARM64 development) hardware, so the two are not yet merged into one table.

Table 3. Classifier-node steady-state cost per backend (batch size 20) and gate cost, single validation run on Jetson hardware.

Node / engine	Throughput (rows/s)	p95 (ms/row)	RSS (MB)	Power (W)
Classifier – ONNX Runtime	20,185	0.0676	155.6	4.64
Classifier – NumPy	152	8.2267	46.2	4.93
Gate – sklearn autoencoder	13,915	0.0783	[TBD]	[TBD]

Power is total-board `tegrastats` power, not idle-subtracted. Gate throughput/latency reported at its own default batch size (100); its forward rate to the classifier topic was 28.64% on the 20,000-record replay sample used for that run, which is not the test split of Sect. 6.6 (11.3%). Source: `submission_checklist.md` in the software repository, single validation run.

6.2 Cold Start: Time to First Verdict

Table 4 breaks down the time from process start to the first served verdict. ONNX Runtime pays a larger one-time session-construction cost (294.5 ms) than NumPy’s artifact load (79.7 ms), but performs its own first prediction fastest (3.1 ms), consistent with the compiled-runtime-vs-interpreted-loop trade-off in Table 3; on this run NumPy still reaches its first verdict sooner overall (368.4 vs. 542.3 ms), since its faster load dominates over the whole warm-up path. This favors NumPy specifically for restart-sensitive scenarios (e.g. frequent container restarts or fast failover) even though ONNX Runtime is far ahead on steady-state throughput.

Table 4. Cold-start latency breakdown per backend (mean of both boards, page cache dropped).

Engine	Load/session build (ms)	First prediction (ms)	Time to first verdict (ms)
ONNX Runtime	294.5	3.1	542.3
NumPy	79.7	115.1	368.4

6.3 NumPy Batch-Size Sensitivity

The original `NumpyInferenceEngine` traverses each tree with its own Python-level while-loop, one tree at a time; because each such loop’s per-call NumPy dispatch

overhead is paid once per tree regardless of how little work that call does, its throughput scales strongly with batch size on Jetson hardware: 9 rows/s at batch 1, 150 rows/s at batch 20, and 1,650 rows/s at batch 500 on the same board (`submission_checklist.md`, this software’s own validation run). ONNX Runtime is comparatively insensitive to batch size over the same range (Sect. 7). This means the two backends should not be compared at a single fixed batch size without stating it, and a deployment choosing the NumPy fallback should size its batch/linger policy accordingly rather than default to a small batch tuned for the ONNX path – or, as Sect. 6.4 shows, the per-tree loop itself is the thing to fix at those batch sizes.

6.4 Block-Lockstep Traversal

The batch-size sensitivity in Sect. 6.3 is a symptom of the per-tree Python loop, not of a NumPy-only backend. Node ids are globally unique across the flattened forest, so nothing requires visiting trees one after another: a $(\text{trees} \times \text{rows})$ matrix of current node ids can be advanced one level per iteration, for every tree and every row at once, cutting the Python-level loop from $O(\text{trees} \times \text{depth})$ iterations to $O(\text{depth})$.

Our first version did exactly that over all 200 trees, and we benchmarked it on synthetic rows. Re-measured on real test-split flows it was 37% *slower* than the per-tree loop at batch size 2,000: the $200 \times 2,000$ node matrix no longer fits in cache, and every level becomes a scattered gather over it. The version reported here advances blocks of about 8,192 (tree, row) pairs at a time, so the working set stays cache-sized at any batch size, and accumulates leaf scores with a gather-and-sum rather than a scatter-add. Table 5 gives throughput on real flows, best of five runs.

Table 5. Per-tree loop vs. block-lockstep traversal on real test-split flows, same artifact (Apple M3, ARM64; not yet re-measured on the Jetson boards).

Batch size	Per-tree loop (rows/s)	Block-lockstep (rows/s)	Speedup
1	78	5,540	71×
20	1,424	15,807	11.1×
100	4,509	16,051	3.6×
500	11,771	16,142	1.4×
2,000	18,177	15,850	0.9×

Source: `code/results/exp1_lockstep_benchmark.json`. Labels match the per-tree loop on 5,000/5,000 rows and ONNX Runtime on 447,275/447,275.

Block-lockstep throughput is nearly flat in batch size, which is the property a streaming node wants: its batches are set by a linger timeout, not chosen for the engine’s convenience (the classifier node defaults to 20, the gate to 100). The per-tree loop overtakes it at batch 2,000, where dispatch overhead is already amortized and its one-tree working set is the most cache-friendly of all; a deployment that really serves such batches should keep the old loop. The rewrite changes no label: it agrees with ONNX Runtime on every row of the test split, and the software’s release gate passes with a maximum confidence difference of exactly zero.

6.5 Exact Early Exit on Gate-Forwarded Traffic

For the binary forest let m be the running margin (attack score minus benign score) after some trees have been evaluated, and let $[\ell_t, u_t]$ be the range of tree t 's leaf margins, read once from the artifact. If $m + \sum_{t \text{ unevaluated}} \ell_t > 0$ the label is attack whatever the remaining trees say; if $m + \sum u_t \leq 0$ it is benign. Rows meeting either condition leave the batch; the rest continue with the next block of trees. Labels are identical to the full forest's by construction. With equally weighted trees no row can qualify before more than half have voted, so the saving is capped at $2\times$; we check first after 104 trees and then every 8.

Table 6. Exact early exit vs. full forest, batch size 2,000 (Apple M3).

Traffic	Rows	Labels identical	Mean trees used	Speedup
Forwarded by the gate	50,548	50,548	104.1	$2.0\times$
All test-split flows	447,275	447,275	105.0	$1.9\times$

Source: `code/results/exp3_exact_early_exit.json`. 99.3% of forwarded rows and 95.2% of all rows stop at the first check.

Table 6 shows the rule running at its ceiling: 99.3% of the flows the gate forwards are settled at the first opportunity. The forest is near-unanimous on the traffic it actually serves, and what limits the saving is the exactness requirement, not the data. Statistical stopping rules [10, 21] would stop far earlier on the same flows, at the price of verdicts that can differ between the primary and the fallback runtime – the property Sect. 4.3 exists to protect.

6.6 What Survives Distribution Shift

Nothing in this paper adapts the detector to drift [7], so it is fair to ask which of its results still hold once traffic stops resembling the training data. We freeze the deployed forest, gate and threshold (all fitted on CICIDS2017) and run them unchanged on the CSE-CIC-IDS2018 test split: 477,771 flows from a different network, year and attack toolset, carrying the same 29 features. This is a cross-dataset shift, harsher than gradual drift, and we use it as a stress test rather than a drift model.

Table 7 separates three kinds of result. *Detection quality collapses*: classifier F1 falls from 0.983 to 0.238, and the anomaly gate, whose threshold is a quantile of 2017 benign traffic, passes under 1% of 2018 attacks while forwarding five times more benign flows than before, so the two-stage pipeline detects almost nothing end to end. Several 2018 attack families (HOIC, Hulk, SSH brute force) are missed by the classifier entirely. This agrees with the cross-dataset failures reported for these datasets and is not something a serving layer can repair. *The serving guarantees are untouched*: the float32 traversal agrees with ONNX Runtime on every shifted flow and on every threshold probe built from shifted rows, and exact early exit returns the full forest's label on all 477,771 flows. They are properties of the model and the arithmetic, not of the traffic. The boundary gate

Table 7. The deployed model and gate, frozen, on in-distribution and shifted traffic.

	CICIDS2017 test	CSE-CIC-IDS2018 test
Flows	447,275	477,771
<i>Detection quality (moves with the data)</i>		
Classifier attack F1 / recall	0.983 / 0.993	0.238 / 0.193
Gate attack recall	0.723	0.009
Gate forward rate (benign only)	0.113 (0.005)	0.025 (0.027)
Two-stage end-to-end attack F1	0.839	0.011
<i>Serving guarantees (do not move)</i>		
NumPy float32 vs. ONNX Runtime, replay flips	0	0
NumPy float32 vs. ONNX Runtime, on-threshold probe flips	0 / 95,741	0 / 95,741
NumPy float64 vs. ONNX Runtime, replay / probe flips	15 / 143	6 / 125
Early-exit labels identical to full forest	447,275	477,771
<i>Early-exit statistics (label-free)</i>		
Mean trees used, all / forwarded	105.0 / 104.1	108.2 / 132.2
Undecided at first check, all / forwarded	4.8% / 0.7%	28.2% / 68.1%

Source: `code/results/exp4_drift.json`. Probe rows take their unconstrained features from the dataset in that column; the probed thresholds are the model’s and do not change.

has a further advantage here: a replay gate is only as current as its recording, whereas the probe set is regenerated from each retrained model’s split table and needs no traffic at all. Note also that replay finds even fewer float64 flips on shifted traffic (6) than in distribution (15); replay-based parity evidence gets weaker exactly when the model is about to be retrained.

The early-exit statistics move, without labels. The share of flows the forest cannot settle at the first check rises from 4.8% to 28.2% (from 0.7% to 68.1% on forwarded traffic): under shift the trees stop agreeing with each other. This is the margin-density signal that Sethi and Kantardzic [17] propose for unlabeled drift detection, and here it is a by-product the serving path already computes. We report it as an observation on one shift; we have not calibrated an alarm threshold or measured its false-alarm rate, and it cannot see drift that leaves the forest confidently wrong, as it is for the missed attack families above.

7 Discussion

The Kafka client, not inference, is now the binding constraint. Once the ONNX Runtime backend is in use, classifier-node throughput (20,185 rows/s) is an order of magnitude above what the pure-Python `kafka-python` client sustains on the same board (roughly 2,500 msg/s consuming, 2,200 msg/s producing; the identical code reaches 6,600/9,900 msg/s on a laptop). Closing this gap is a client-library problem, not an inference problem: a C-backed client such as `confluent-kafka` is the next lever, and reporting an end-to-end (producer-timestamp-to-verdict) throughput number without controlling for this would misattribute a Kafka-client ceiling to the classifier.

ONNX vs. NumPy is a genuine trade-off, not a strict ordering. Table 3 makes ONNX Runtime look like the only reasonable choice, but Table 4 and the artifact-size

row of Table 1 complicate that: the NumPy artifact is $6\times$ smaller and reaches its first verdict sooner, which matters on boards where `onnxruntime` cannot be installed at all, or where restart frequency dominates steady-state throughput. The toolkit’s role is to make that trade-off a one-flag choice rather than a rewrite.

What replay-based parity gates miss. The software’s release gate replays 2,000 rows and had never failed. Sect. 4.3 shows that this says little: a traversal that disagrees with ONNX Runtime at split boundaries passes such a gate 93.5% of the time. The practical change is small – derive the probe set from the artifact’s own split table and run it alongside the replay – and because the forest has only 815 distinct thresholds, an exhaustive probe of every distinct boundary value is cheaper than the replay it complements. We expect the threshold sharing, and therefore both the label-flip risk and the cheap remedy, to hold for any forest trained with binned candidate splits, but we have measured one forest and do not claim more.

Lockstep traversal and the lesson from ARM tree-ensemble work. Koschel et al. [11] obtain their speedups by evaluating many trees per instruction with NEON bitvectors. We took only the underlying lesson – stop walking trees one at a time – and applied it at the NumPy level. Table 5 is the result, including the first attempt’s failure: whole-forest lockstep looked good on synthetic rows and lost to the old loop on real ones at large batches, until the traversal was blocked to cache size. NEON intrinsics and fixed-point thresholds remain a separate, larger step.

Quantizing the ONNX graph itself. Eren et al. [5] demonstrate that ONNX-level quantization measurably changes throughput and latency for network-security classifiers, on DNN architectures rather than tree ensembles. Whether the same holds for a `TreeEnsembleClassifier` graph (whose costs are dominated by branching and memory access, not the matrix multiplies quantization typically targets) is an open, testable question for this toolkit rather than an assumption we import; we list it as future work.

Limitations. The on-board figures (Tables 3, 1, 4) come from a single validation run with Jetson #2 on Wi-Fi, and their NumPy rows describe the per-tree traversal that was deployed at the time. Tables 5 and 6 were measured on an ARM64 development machine, not on the boards; *[re-measure both on the Jetson Cortex-A78AE boards and merge into Table 3]*. Disagreement and label-identity counts (Tables 2, 6) are machine-independent for the NumPy engine but were taken against one ONNX Runtime build (1.30, CPU provider); a different provider is a different runtime and needs its own probe run. Power is total-board `tegrastats` power without an idle baseline, so it is not per-inference energy. End-to-end latency over a wired inter-board link is not yet measured; *[fill in once available]*. The boundary probes cover axis-aligned `BRANCH_LEQ` splits of one forest; they do not exercise missing-value routing or other split modes. Finally, the detector itself is static. On its own test split the deployed gate forwards only 72.3% of attacks, which bounds end-to-end recall however the classifier is served, and under the shift of Sect. 6.6 both gate and classifier fail; this paper measures that failure but offers no adaptation, and the undecided-rate signal is an uncalibrated observation on a single shift.

8 Conclusion

A detector that can be served by two runtimes is only as trustworthy as the evidence that they agree. For ONNX-EDGEIDS that evidence was a 2,000-row replay, and we showed it to be weak: a float64 traversal that flips 328 labels on boundary-directed probes flips 15 on the entire 447,275-row test split, and would pass the replay gate 93.5% of the time. Probing every split threshold of the deployed forest – 287,223 inputs, 99.9% of which reach their target node – turns the question into one with a deterministic answer, and for the float32 traversal we deploy the answer is zero disagreements. The flips reach the label because 95,741 splits share 815 thresholds, which also makes the exhaustive check cheap. Around that result, a block-lockstep rewrite makes the NumPy fallback usable at streaming batch sizes ($71\times$ at batch 1, $11\times$ at 20, and slower than the old loop only at 2,000), and an exact early-exit rule halves its work without being able to change a verdict. None of this makes the detector robust to drift: replayed on CSE-CIC-IDS2018 the frozen model and gate collapse, while parity and early-exit exactness hold on every shifted flow and the early-exit undecided rate rises sixfold without labels. On the boards ONNX Runtime remains far ahead on throughput while the NumPy artifact is $6\times$ smaller and starts 32% faster, and once ONNX Runtime is in use the Python Kafka client, not inference, is the bottleneck. *[Before submission: re-measure Tables 5 and 6 on the Jetson boards, measure wired-link end-to-end latency, and repeat the single-run on-board figures.]* Beyond that: calibrating the undecided-rate signal as a retraining trigger, boundary probes for other ONNX Runtime providers and split modes, NEON-level traversal [11], ONNX-level quantization of the tree graph [5], and a C-backed Kafka client.

Disclosure of Interests. The authors declare that they have no competing interests.

References

1. Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., Rieck, K.: Dos and don'ts of machine learning in computer security. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 3971–3988 (2022)
2. Chithra Rani, P.R., Baalaji, K.: Deep learning-based ensemble stacking for enhanced intrusion detection in IoT-edge platforms. *Discover Applied Sciences* **7**, 928 (2025). <https://doi.org/10.1007/s42452-025-06871-z>
3. Diaz-Gorin, J., Caballero-Gil, C., Brankovic, L.: Enhancing network security with generative AI on Jetson Orin Nano. *Applied Sciences* **16**(3), 1442 (2026). <https://doi.org/10.3390/app16031442>
4. Duan, B., Dong, R., Dong, N., Kim, D.D., Yang, G.: Testing deep learning library APIs via cross-framework differential fuzzing (2026), arXiv:2608.11886
5. Eren, S., Gültekin, A., Özkan, Ö., Özçelik, İ.: High-throughput and low-latency DrDoS detection using quantized ONNX models. *Symmetry* **18**(7), 1187 (2026). <https://doi.org/10.3390/sym18071187>
6. Ferreira, L., Abreu, R., Branco, F., Reis, M.J.C.S., Serôdio, C., Valente, A.: A two-stage hybrid intrusion detection system for CAN bus based on statistical thresholds and random forest classifiers. *Electronics* **15**(11), 2239 (2026). <https://doi.org/10.3390/electronics15112239>

7. Gama, J., Zliobaitė, I., Bifet, A., Pechenizkiy, M., Bouchachia, A.: A survey on concept drift adaptation. *ACM Computing Surveys* **46**(4), 1–37 (2014)
8. Hamdan, S., Ayyash, M., Almajali, S.: Edge-computing architectures for internet of things applications: A survey. *Sensors* **20**(22), 6441 (2020). <https://doi.org/10.3390/s20226441>
9. Hasan, T., Hossain, A., Ansari, M.Q., Syed, T.H.: Enhanced intrusion detection in IIoT networks: A lightweight approach with autoencoder-based feature learning (2025), [arXiv:2501.15266](https://arxiv.org/abs/2501.15266)
10. Hernández-Lobato, D., Martínez-Muñoz, G., Suárez, A.: Statistical instance-based pruning in ensembles of independent classifiers. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **31**(2), 364–369 (2009). <https://doi.org/10.1109/TPAMI.2008.204>
11. Koschel, S., Buschjäger, S., Lucchese, C., Morik, K.: Fast inference of tree ensembles on ARM devices (2023), [arXiv:2305.08579](https://arxiv.org/abs/2305.08579)
12. Lanvin, M., Gimenez, P.F., Han, Y., Majorczyk, F., Mé, L., Totel, É.: Errors in the CI-CIDS2017 dataset and the significant differences in detection performances it makes. In: *Risks and Security of Internet and Systems (CRiSIS 2022)*, Revised Selected Papers. pp. 18–33. *Lecture Notes in Computer Science*, Springer (2023). https://doi.org/10.1007/978-3-031-31108-6_2
13. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems* **30** (2017)
14. Naik, A., Garg, C., Akshay, S., Gupta, A., Meel, K.S.: Quantifying sensitivity for tree ensembles: A symbolic and compositional approach (2026), [arXiv:2605.13830](https://arxiv.org/abs/2605.13830)
15. Nakandala, S., Saur, K., Yu, G.I., Karanasos, K., Curino, C., Weimer, M., Interlandi, M.: A tensor compiler for unified machine learning prediction serving. In: *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*. USENIX Association (2020)
16. Nasir, K., Badri, S.K., Alghazzawi, D.M., Alghamdi, M.Y., Alkhozai, M., Almakky, A., Alhazmi, R.M., Asghar, M.Z.: HED-ID: an edge-deployable and explainable intrusion detection system optimized via metaheuristic learning. *Scientific Reports* **16**, 2313 (2026). <https://doi.org/10.1038/s41598-025-32183-8>
17. Sethi, T.S., Kantardzic, M.: On the reliable detection of concept drift from streaming unlabeled data. *Expert Systems with Applications* **82**, 77–99 (2017). <https://doi.org/10.1016/j.eswa.2017.04.008>
18. Sforzin, A., Gómez Mármol, F., Conti, M., Bohli, J.M.: RPiDS: Raspberry pi IDS — a fruitful intrusion detection system for IoT. In: *Proc. Int. IEEE Conf. on Ubiquitous Intelligence & Computing (UIC/ATC/ScalCom)*. pp. 440–448 (2016). <https://doi.org/10.1109/UIC-ATC-ScalCom-CBDCoM-IoP-SmartWorld.2016.0080>
19. Sharafaldin, I., Lashkari, A.H., Ghorbani, A.A.: Toward generating a new intrusion detection dataset and intrusion traffic characterization. In: *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*. pp. 108–116 (2018)
20. Shi, W., Cao, J., Zhang, Q., Li, Y., Xu, L.: Edge computing: Vision and challenges. *IEEE Internet of Things Journal* **3**(5), 637–646 (2016). <https://doi.org/10.1109/JIOT.2016.2579198>
21. Wang, S., Gupta, M., You, S.: Quit when you can: Efficient evaluation of ensembles with ordering optimization (2018), [arXiv:1806.11202](https://arxiv.org/abs/1806.11202)
22. Zhang, Z., Zhang, L., Xu, J., Chen, Z., Wang, P.: An edge-deployable lightweight intrusion detection system for industrial control. *Electronics* **15**(3), 644 (2026). <https://doi.org/10.3390/electronics15030644>